

Public, Private and Class Attributes, Class and Static Methods

Introduzione

Come abbiamo già introdotto nella scorsa lezione dell'[Ereditarietà delle classi](#) al capitolo [Gli attributi protetti](#), un attributo può essere protetto tramite la convenzione di naming del singolo underscore(`_`):

Un attributo protetto è inteso per essere accessibile solo all'interno della classe che lo definisce e alle sue sottoclassi.

Tuttavia in Python bisogna distinguere i diversi tipi di attributo o per meglio dire le diverse modalità di accesso agli attributi.

Gli attributi pubblici

Un attributo è pubblico quando è accessibile da qualsiasi punto nel codice: sia dentro le classi padre che figlie sia al di fuori delle classi.

Per definirlo basta non inserire nessun underscore prima del nome.

Ciò significa che sono attributi completamente accessibili e modificabili e vengono usati quando si vuole esporre il comportamento o il dato senza restrizioni.

Ovviamente questo comporta dei pericoli è pericoloso perché rende l'attributo accessibile, da chiunque nel codice, e, soprattutto, modificabile.

Esempio:



Python

```
1 class Animal:
2     def __init__(self) -> None:
3         self.name:str = "Generic Animal" # Public attribute
4
5 animal:Animal = Animal()
6 print(animal.name) # Output: "Generic Animal"
7 animal.name = "It's a dog" # MODIFIES the public attribute 'name'
8 print(animal.name) # Output: "It's a dog"
```

nel primo print è "generic animal" ma nel secondo caso si assegna un nuovo valore stringa ("It's a dog") all'attributo `name` , questo è pericoloso poiché l'attributo può essere facilmente preso e modificato da qualsiasi user e/o sviluppatore.

Rischio degli attributi pubblici

nessuna protezione

nessun controllo

può portare a bugs

un alternativa è usare attributi privati o accedere via getter e setter per mantenere il controllo.

Attributi privati

Un attributo è privato quando:

è nascosto e non è accessibile direttamente dall'esterno della classe.

A differenza dei linguaggi che impongono veri e propri meccanismi di privacy (come `private` in Java o C#), Python utilizza una convenzione basata sul **name mangling**:

in sostanza Python rinomina internamente l'attributo in

`_NomeClasse__nome__attributo` **per nascondere.**

Ciò rende più difficile (ma non impossibile) l'accesso agli attributi privati, poiché rende gli attributi difficili da accedere accidentalmente ma non è impossibile raggiungerli deliberatamente.

Per definire un attributo privato si antepongono un **doppio underscore** (`__`), ad esempio (`__nomeAttributo`).

Un attributo è privato è sicuramente nascosto rispetto all'attributo protetto.

Tuttavia l'attributo privato viene rinominato attraverso il **name mangling**, quindi in realtà non è privato ma è nascosto nel codice.

Infatti questo tipo di attributi poiché non sono veramente protetti vi si può accedere forzatamente .

Esempio con attributi privati:



Python

```
1 class Animal:
2     def __init__(self) -> None:
3         self.__name:str = "Generic Animal" # Attributo Privato
4
5 animal:Animal = Animal()
6 print(animal.name) # Error: 'Animal' object has no attribute
   'name'
```

```
7 animal.name = "It's a dog" # Crea un nuovo attributo pubblico
  'name'
8 print(animal.name) # Output: "It's a dog"
```

Come si può vedere:

- Il primo `print(animal.name)` : genera un **errore** perché l'attributo `__name` non è accessibile direttamente.
- Quando si fa `animal.name = "It's a dog"` , **non stiamo modificando l'attributo privato `__name` , bensì creiamo un nuovo attributo pubblico chiamato `name` .**

Come accennato poco sopra si può anche forzare l'attributo privato facendo così:



Python

```
1 class Animal:
2     def __init__(self) -> None:
3         self.__name:str = "Generic Animal" # Attributo Privato
4
5 animal:Animal = Animal()
6 animal.name = "It's a dog" # Crea un nuovo attributo pubblico
  'name'
7 print(animal.name) # Output: "It's a dog"
8 print(animal._Animal__name) # Forzo l'accesso all'attributo
  privato name
```

In questo caso stiamo forzando l'accesso all'attributo `name` contenuto nella classe `Animal` , difatti l'output sarà `'Generic Animal'` .

🚫 Questa pratica di forzare l'accesso agli attributi privati è fortemente sconsigliata: oltre a rompere l'incapsulamento può rendere il codice fragile e difficile da mantenere

Accesso agli attributi Privati: Getter e Setter

Quindi, in Python, per evitare di accedere al valore dell'attributo in maniera forzata si usano, per leggere o modificare un attributo privato dall'esterno della classe, i metodi getter e setter:

- Il **Getter**: restituisce il valore dell'attributo

- Il **Setter**: modifica il valore dell'attributo

Questi due metodi permettono di gestire gli attributi in maniera controllata .

Esempio con il `get` :



Python

```
1  Class Animal:
2      def __init__(self) -> None:
3          self.__name:str = "Generic Animal" # Attributo privato
4      # Rende 'name' accessibile
5      def get_name(self) -> str:
6          return self.__name
7
8  animal:Animal = Animal()
9  print(animal.get_name()) # Output: "Generic Animal"
```

Quindi se l'utente chiama `get_name` , legge il valore dell'attributo.

Se si volesse anche modificare il valore dell'attributo `name` si deve aggiungere un **setter**.

Esempio con il `set` :



Python

```
1  class Animal:
2      def __init__(self) -> None:
3          self.__name: str = "Generic Animal"
4
5      # Getter
6      def get_name(self) -> str:
7          return self.__name
8
9      # Setter
10     def set_name(self, new_name: str) -> None:
11         self.__name = new_name
12
13  animal: Animal = Animal()
14  print(animal.get_name()) # Output: "Generic Animal"
15  animal.set_name("Dog")
16  print(animal.get_name()) # Output: "Dog"
```

🔖 Quindi per gli attributi privati uso un getter per leggere il valore e un setter per modificarlo.

👁 Info

Di norma, **getter e setter** vengono sempre implementati insieme per gestire correttamente gli attributi privati.

Gli attributi di classe e i metodi statici

I metodi statici

Come abbiamo già ampiamente introdotto nella lezione [Python/Lezione 6__Le Classi__ Gli attributi pubblici,privati, gli attributi di classe e i metodi di classe/Le Classi](#), al capitolo [Utilizzo dei Decoratori**`@property`**per Getter e Setter](#): il metodo statico: si definisce con l'operatore chiocciola (`@staticmethod`).

- **Non riceve né `self` né `cls`** come primo parametro, quindi non ha accesso né agli attributi dell'istanza né a quelli della classe.
- Può essere chiamato **senza creare un'istanza** della classe.
- È completamente **indipendente** sia dai dati dell'istanza che della classe: il metodo non interagisce con l'oggetto o la classe stessa.

In pratica, **un metodo statico è un metodo che, pur facendone parte, non ha bisogno di accedere a dati o comportamenti specifici della classe o delle sue istanze.**

Quando usare un metodo statico?

Un metodo statico è utile quando:

- **Il metodo ha senso essere raggruppato nella classe (per motivi logici o organizzativi)** ,
- **Ma non necessita di accedere né agli attributi dell'istanza (`self`) né a quelli della classe (`cls`)** .

Esempio:



Python

```
1 class MathTools:
```

```

2     @staticmethod
3     def add(a: int, b: int) -> int:
4         return a + b
5
6     # Uso del metodo statico
7     result = MathTools.add(3, 5)
8     print(result) # Output: 8

```

In questo caso:

- Non serve creare un oggetto `MathTools` per usare `add()`.
- `add()` è indipendente dalla classe `MathTools` e non interagisce con gli attributi della classe.

Chiamata da Istanza

Anche se il metodo è statico, può comunque essere chiamato su un'istanza:

```

1 math_tool = MathTools()
2 print(math_tool.add(10, 20)) # Funziona lo stesso! Output: 30

```

Anche se chiamato su un oggetto, il comportamento rimane identico perché il metodo è "statico" e non dipende dai dati dell'istanza.

 I metodi statici sono ideali quando il comportamento appartiene logicamente alla classe, ma non dipende da nessun dato dell'istanza o della classe stessa. Vanno usati quando non è necessario mantenere uno stato interno legato all'oggetto.

Gli attributi della classe

Come abbiamo ampiamente detto nella lezione [Python/Lezione 6_ Le Classi_ Gli attributi pubblici, privati, gli attributi di classe e i metodi di classe/Le Classi](#), al capitolo [Attributi di classe](#); gli **attributi di classe** sono variabili che appartengono **alla classe stessa**, non alle sue istanze.

Questo significa che sono **condivisi** tra **tutte le istanze**: ogni oggetto vede e può usare lo stesso valore, ma l'attributo **rimane collegato alla classe**, non al singolo oggetto.

Esempio attributi di classe:



```

1  class Person:
2      population:int = 0 # Public class attribute
3      def __init__(self, name:str) -> None:
4          self.__name:str = name # Private attribute
5          Person.population += 1
6
7
8  print(Person.population) # Output: 0
9  person1:Person = Person("Alice")
10 print(Person.population, person1) # Output: 1
11 person2 = Person("Bob")
12 print(Person.population, person2) # Output: 2

```

Spiegazione:

- `population` è definito **nella classe** `Person` , **fuori** da ogni metodo.
→ È un **attributo di classe**.
- `self.__name` è invece un **attributo di istanza**, perché è legato a ogni singolo oggetto `Person` .
- Ogni volta che creiamo un nuovo oggetto (`Person("Alice")` , `Person("Bob")`), il costruttore (`__init__`) incrementa `Person.population` di **1**.
- Quindi `population` tiene traccia **globale** di quante persone sono state create.

Esempio con metodi statici:



Python

```

1  class Math:
2      PI:float = 3.14 # Public class attribute
3
4      @staticmethod
5      def circle_area(radius) -> float: # Static Method
6          return Math.PI * radius * radius
7
8
9  print(Math.PI) # Output: 3.14
10 print(Math.circle_area(5)) # Output: 78.5

```

Qui manca il `self` perché non è legato all'istanza, infatti nel primo print mi stampa l'attributo di classe e nel secondo print mi stampa il risultato del calcolo dell'area di

un cerchio.

io ho usato il metodo `circle_area` senza che ho creato l'istanza di `Math`

Metodo di classe

un class method quindi è un metodo legato non all'istanza ma alla classe e come parametri di definizione prende `cls` e non `self`, perché fa riferimento agli attributi di classe al fine di manipolarli.

Sempre come abbiamo visto nel capitolo [Attributi di classe](#) il metodo di classe:

- È definito con `@classmethod`
- Prende `cls` (non `self`) come primo parametro.
- Può accedere e modificare gli attributi a livello di classe.

Esempio:



Python

```
1  class Person:
2      __population:int = 0 # Private class attribute
3      def __init__(self, name:str) -> None:
4          self.__name:str = name # Private attribute
5          Person.__population += 1
6      @classmethod
7      def get_population(cls):
8          return cls.__population
9
10     print(Person.get_population()) # Output: 0
11     person1 = Person("Alice")
12     print(Person.get_population()) # Output: 1
13     person2 = Person("Bob")
14     print(Person.get_population()) # Output: 2
```